20 van Emde Boas 樹

Part V Advanced Data Structures

本章定位

van Emde Boas樹利用整數鍵的高低位元，快速找前驅與後繼。雙對數時間的關鍵是min/max快取讓每層只走一條昂貴遞迴；最小值獨立儲存與線性universe配置成本則決定更新規則和使用條件。

前置知識

整數字 universe、位元分割與遞迴摘要。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 比較 bitmap、summary 與 clustered 結構。
- 解釋 vEB 遞迴結構的 universe 與 summary。
- 推導 insert、successor 與 min/max 時間。
- 20.1 Preliminary approaches
- 20.2 A recursive structure
- 20.3 The van Emde Boas tree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

20.1 Preliminary approaches

整數集合的universe為U={0,⋯,u−1}，u=2^b且b≥1。successor(x)指嚴格大於x的最小集合元素，predecessor則嚴格小於；x不必已在集合中，沒有答案時回NIL。

本章採word-RAM，字長w≥b，合法鍵、位移、遮罩及陣列索引各花O(1)。這不是逐位運算的bit-complexity模型；若一個鍵跨多個機器字，不能沿用同一常數。

bitmap用u位，member可直接讀位元。但successor一般要往後掃；即使每字可快速找第一個1，最壞仍可能掃Θ(u/w)字。只有整個universe放一字且模型提供相應bit-scan時，才可把它當O(1)。紅黑樹則以O(n)節點、O(log(n+1))時間處理任意比較鍵，無需先配置整個u。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分群可以省掃描，但不自動得雙對數

u=256可分16群，每群16鍵。summary中第i位為1代表群i非空。找successor時，先判斷本群是否有更大鍵，再找後續非空群，最後取該群最小鍵。

若兩層都採逐位掃描，最壞工作約群內16加summary16，仍依群大小增加。若群寬m，粗略最壞界為O(m+u/m)，在m≈√u時為O(√u)，不是O(log log u)。

改成多層遞迴後，還必須數清楚每層到底有幾個昂貴子呼叫。下一節的proto結構示範「每次universe開根號」本身不足以保證目標時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

20.2 A recursive structure

把b個位元拆成上半⌈b/2⌉位與下半⌊b/2⌋位。設

```
upper=2^ceil(b/2); lower=2^floor(b/2)
high(x)=floor(x/lower); low(x)=x mod lower
index(h,l)=h*lower+l
```

有upper個cluster，每個universe大小lower；summary自身的universe大小upper，保存非空cluster的索引。不是在所有地方都把兩種大小一律寫成同一個√u。

u=32時b=5、upper=8、lower=4。x=27拆成high=6、low=3，再組回6×4+3=27。若把兩者都取√32再四捨五入，會失去完整且不重疊的位元分割。

Proto-vEB 的兩個遞迴為何仍慢

先看b為2冪、可恰好等分的情形。沒有快取min/max的proto-vEB，求最小值要遞迴找summary的最小非空群，再遞迴找該群最小鍵：

$T_{min}(u)=2T_{min}(\sqrt{u})+\Theta(1)$。

令S(b)=T(2^b)，得S(b)=2S(b/2)+Θ(1)=Θ(b)，即Θ(log u)。插入若每次都同時遞迴cluster與summary，也有此界。

proto-successor可能先遞迴本群、失敗後遞迴summary，再求新群minimum，所以最壞

$T_{succ}(u)=2T_{succ}(\sqrt{u})+\Theta(\log u)$。

換元得S(b)=2S(b/2)+Θ(b)=Θ(b log b)，即Θ(log u log log u)。改進必須減少每層的昂貴分支，不能只引用遞迴深度。

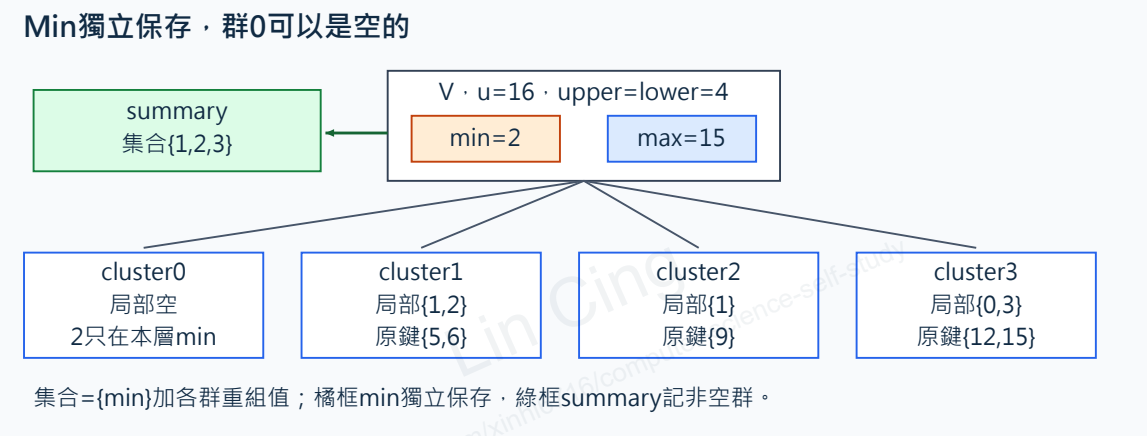
20.3 The van Emde Boas tree

每個vEB節點額外保存min、max，空集合兩者皆NIL。min不放入任何cluster；除min外的所有鍵依high/low唯一放入cluster。summary只保存這些cluster中非空者的索引。

因此單元素時min=max且所有cluster空；至少兩元素時max也在某個cluster中。u=2是基底，只須min/max，沒有再配置cluster。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

20.3 The van Emde Boas tree （續）



數例u=16、S={2,5,6,9,12,15}：min=2、max=15；cluster0空，cluster1存局部{1,2}，cluster2存{1}，cluster3存{0,3}；summary={1,2,3}。不能因high(2)=0就強迫summary包含0，因2只在本層min。

空、單元素與Member

```
member(V,x):
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return member(V.cluster[high(x)],low(x))
minimum(V): return V.min
maximum(V): return V.max
empty-insert(V,x): V.min=V.max=x
```

member每層只進一個cluster，且universe嚴格變小。min/max與空/單元素判斷皆O(1)；empty-insert不用遞迴填整棵樹，因新min本來就不屬於cluster。這個常數基底也會讓更新只剩一條昂貴遞迴。

公開介面先確認x是0..u−1整數。INSERT已存在鍵、DELETE不存在鍵可先以member偵測，回報false而不更動集合；加一次同階查詢不改變漸近時間。下列內部更新偽碼採「鍵尚未存在／鍵確實存在」前提。

Successor 的互斥分支

```
successor(V,x):
    if V.min==NIL or x>=V.max: return NIL
    if x<V.min: return V.min
    if V.u==2: return 1
    h=high(x); l=low(x); C=V.cluster[h]
    if C.max!=NIL and l<C.max:
        return index(h,successor(C,l))
    j=successor(V.summary,h)
    if j==NIL: return NIL
    return index(j,V.cluster[j].min)
```

先讀C.max就能知道同群是否存在更大值。如果存在，只遞迴C；否則直接遞迴summary，不先花時間搜尋已知不會成功的C。最後讀下一群min是O(1)，所以每層至多一個昂貴子呼叫。

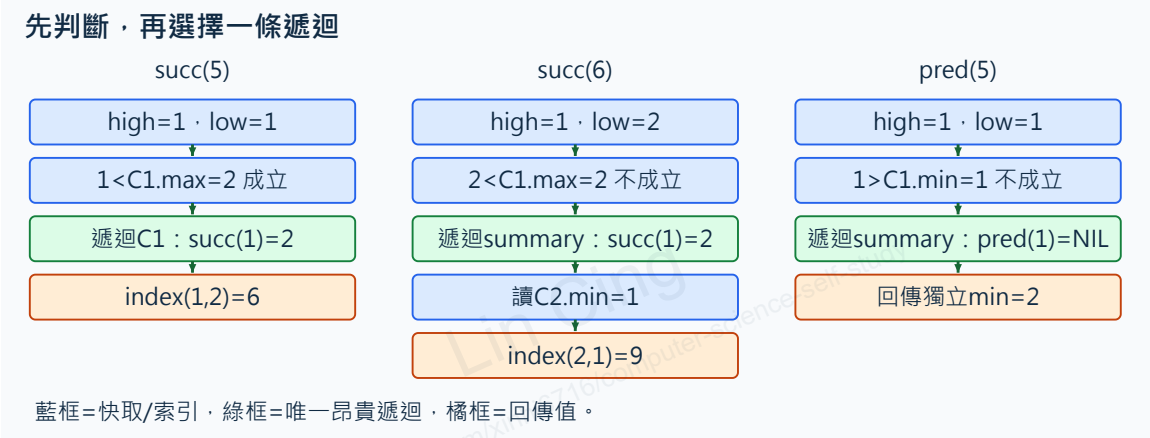
Predecessor 必須補回全域Min

```
predecessor(V,x):
    if V.min==NIL or x<=V.min: return NIL
    if x>V.max: return V.max
    if V.u==2: return 0
    h=high(x); l=low(x); C=V.cluster[h]
    if C.min!=NIL and l>C.min:
        return index(h,predecessor(C,l))
    j=predecessor(V.summary,h)
    if j==NIL: return V.min
    return index(j,V.cluster[j].max)
```

到j=NIL分支時，前面已確認x>V.min，所以全域min仍是合法答案。不能把successor的NIL分支完全鏡射，因min刻意沒有放在cluster裡。

例S={2,5,6,9,12,15}，pred(5)在cluster1內沒有更小局部鍵，summary也沒有比1更小的群，仍須回2。

同群與跨群的實際查詢



succ(5)：high=1、low=1，cluster1.max=2，1<2成立，只遞迴cluster1，得到局部2，組回1×4+2=6。

succ(6)：high=1、low=2，不小於cluster1.max=2，所以不遞迴cluster1。summary的succ(1)=2，讀cluster2.min=1，組回2×4+1=9。

同群與跨群的實際查詢 (續)

查詢值不必屬於S。succ(3)也走cluster0空的跨群分支，得到cluster1.min=1，組回5；succ(15)=NIL，pred(2)=NIL。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insert 的Min交換與空群分支

```
insert-new(V,x):
    if V.min==NIL: empty-insert(V,x); return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        h=high(x); l=low(x); C=V.cluster[h]
        if C.min==NIL:
            insert-new(V.summary,h)
            empty-insert(C,l)
        else: insert-new(C,l)
    if x>V.max: V.max=x
```

若新x比min小，交換後新x直接留在min，原min成為接下來要插入cluster的值。空群分支雖更新summary與cluster兩處，但empty-insert是O(1)；非空群只遞迴cluster，不動summary，所以仍至多一條昂貴遞迴。

例向上述S插1：min變1，原min2送入cluster0的局部2，summary新增0。最後min=1、max=15，cluster0={2}，而不是把新min1也放進cluster0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Delete 的基底與最小值提升

```
delete-present(V,x):
    if V.min==V.max: V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=(1 if x==0 else 0); return
    if x==V.min:
        h=V.summary.min
        x=index(h,V.cluster[h].min)
        V.min=x
    h=high(x); l=low(x); C=V.cluster[h]
    delete-present(C,l)
```

此程序接著執行下一頁的summary/max收尾區塊。多元素時刪min，必須先找cluster中最小的元素提升成新min，然後從原cluster刪掉這個提升值，保持min不重複儲存。此處變數x改為提升值，已不再是呼叫者原本的舊min。

Delete 的Summary與Max完整收尾

接續前頁的C刪除：

```
if C.min==NIL:
    delete-present(V.summary,h)
    if x==V.max:
        j=V.summary.max
        if j==NIL: V.max=V.min
        else: V.max=index(j,V.cluster[j].max)
elif x==V.max:
    V.max=index(h,C.max)
```

若C刪後為空，表示刪前只有單一元素，剛才的delete-present(C,l)只做O(1)基底工作，才會接著遞迴summary。若cluster刪除真的沿多層走，C不會變空，就不會再遞迴summary。兩個昂貴分支仍互斥。

若不更新summary，日後可能走進空群；若不更新max，successor可能因錯誤的上界判斷而選錯分支，這兩欄不是可有可無的快取。

由同一資料重算更新結果

從S={2,5,6,9,12,15}出發：

操作後	min	max	cluster0	cluster1	cluster2	cluster3	summary
原集合	2	15	空	{1,2}	{1}	{0,3}	{1,2,3}
insert1	1	15	{2}	{1,2}	{1}	{0,3}	{0,1,2,3}
delete1	2	15	空	{1,2}	{1}	{0,3}	{1,2,3}
delete15	2	12	空	{1,2}	{1}	{0}	{1,2,3}

delete1先把cluster0的2提升到min，再刪局部2，群0空才刪summary中的0。delete15後群3仍有局部0，故summary保留3，新max=3×4+0=12。每列都可由min與所有cluster組回完整集合，沒有遺失或重複提升值。

正確性與雙對數時間

用b歸納，基底u=2由min/max精確代表空、單元素或{0,1}。假設較小universe的操作正確，每層用互斥high範圍分群，index是high/low的反函數，min再獨立保存。

member只到唯一可能的群；successor/predecessor以群max/min判斷是否能在本群找到，否則用summary找最近群，並另處理獨立min。insert交換與delete提升都只移轉一個代表，summary恰與群非空條件同步，所以表示不變量與集合操作皆維持。

每操作最多一個非O(1)子呼叫，其bit數≤⌈b/2⌉。因此

`T(2^b)≤T(2^ceil(b/2))+O(1)`。

對b≥2，⌈b/2⌉≤2b/3，至b=1只需O(log b)層，時間O(1+log log u)。這個結論依賴前面逐項證明的互斥分支，不能單靠樹的層數。

預先配置的空間遞迴

原版預先建立所有summary與cluster，空集合也佔Θ(u)字，初始化Θ(u)時間。令P(u)包含節點欄位與cluster指標陣列，u>2時

`P(2^b)=2^ceil(b/2) P(2^floor(b/2))+P(2^ceil(b/2))+Θ(2^ceil(b/2))`。

設F(b)=P(2^b)/2^b、ε_b=2^(−floor(b/2))，則對固定c，

`F(b)≤F(floor(b/2))+ε_b F(ceil(b/2))+c ε_b`。

取G=F+c，可得 `G(b)≤(1+ε_b) max(G(floor(b/2)),G(ceil(b/2)))`。接著沿較大那項反覆下降到b=1，可把整個倍率界成有限乘積，而不把summary成本忽略掉。

空間線性界及配置模式的限制

沿上述halving路徑從基底反向數r層，bit數至少2^(r−1)+1，因此ε依序至多1/2、1/2、1/4、1/16、…，總和有與b無關的常數界。由1+x≤e^x，

`∏(1+ε)≤exp(Σε)=O(1)`，

所以F(b)=O(1)，P(u)=O(u)。只走cluster分支到u=2，就已把原universe分成u/2個基底節點，給Ω(u)，合得Θ(u)。初始化寫入這些欄位同階Θ(u)。

若u很大而n很小，預先配置可能完全不划算。改成按需建立群與hash dictionary時，建群成本、dictionary時間與空間都需要另證；不能只寫「lazy」就同時承諾O(n)空間及所有操作最壞O(log log u)。本章已證的界明確限於預先配置、直接陣列索引的word-RAM版本。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 20。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study